

Documentación del código — Test 1 (T_n , Schuster–Barker)

Proyecto 2: Estadística No Paramétrica

Equipo: Gabriel Sánchez, Juan David Duarte, Luis Tejón, Claudio (IA)

Mayo 2026

*Este documento explica el código implementado para el **Test 1** del Proyecto 2: el test bootstrap de simetría univariada basado en el estadístico tipo Kolmogorov–Smirnov de Schuster & Barker (1987). Cubre la motivación matemática, la estructura del repositorio, la descripción de cada módulo y las instrucciones para reproducir las simulaciones.*

Índice

1. Visión general	2
1.1. Tres estimadores del centro de simetría	2
1.2. Diseño experimental	2
2. Estructura del repositorio	2
3. Módulos del paquete <code>src/</code>	3
3.1. <code>statistics_ks.py</code> — estadístico T_n y estimadores	3
3.2. <code>bootstrap_ks.py</code> — procedimiento bootstrap	4
3.3. <code>distributions.py</code> — generadores de muestras	4
3.4. <code>simulation.py</code> — estudio Monte Carlo secuencial	5
3.5. <code>simulation_parallel.py</code> — versión paralelizada	5
3.6. <code>plotting.py</code> — visualizaciones	5
4. Scripts ejecutables	6
4.1. Versión secuencial	6
4.2. Versión paralelizada (recomendada)	6
5. Salidas generadas	6
5.1. Archivos de datos	6
5.2. Figuras generadas	7
6. Tests unitarios	7
7. Dependencias e instalación	7
7.1. Python	7
7.2. Paquetes	7
8. Reproducir la simulación completa — guía paso a paso	8
9. Resumen de resultados obtenidos	8

1 Visión general

El Test 1 evalúa la hipótesis nula

$$H_0 : F \text{ es simétrica respecto a algún } \theta,$$

frente a la alternativa $H_a : F$ no simétrica, usando el estadístico

$$T_n(\tilde{\theta}) = \sqrt{n} \|F_n - sF_n(\cdot; \tilde{\theta})\|_{\infty},$$

donde F_n es la cdf empírica de la muestra y $sF_n(\cdot; \theta)$ es su *simetrización* respecto a θ , es decir, la cdf empírica de los $2n$ puntos $\{X_1, \dots, X_n, 2\theta - X_1, \dots, 2\theta - X_n\}$.

Dado que la distribución de T_n bajo H_0 depende de la F desconocida, se usa el procedimiento bootstrap de Schuster–Barker: se generan B remuestras de $sF_n(\cdot; \tilde{\theta})$ y se aproxima el p-valor con la proporción de réplicas que superan el estadístico observado.

1.1 Tres estimadores del centro de simetría

- **argmin:** $\tilde{\theta} = \arg \min_{\theta} T_n(\theta)$ (minimiza la propia norma KS).
- **mediana:** $\tilde{\theta} = \text{median}(X_1, \dots, X_n)$.
- **media afeitada:** $\tilde{\theta} = \bar{X}_{\alpha}$ con $\alpha = 0,1$ por defecto.

1.2 Diseño experimental

- Distribuciones bajo H_0 : Uniforme(1,3), Cauchy(loc=2, scale=1) y opcionalmente Normal.
- Distribuciones bajo H_a : Gamma(2,1), Weibull(1.5,1), Pareto(3,1).
- Tamaños muestrales: $n \in \{20, 40, 80, 160\}$.
- Réplicas bootstrap: $B = 500$.
- Réplicas Monte Carlo: $R = 300$ por celda.
- Nivel: $\alpha = 0,05$.

2 Estructura del repositorio

```
Proyecto2EstadisticaNP/
|-- src/
|   |-- __init__.py
|   |-- statistics_ks.py           # T_n y estimadores del centro
|   |-- bootstrap_ks.py           # Procedimiento bootstrap
|   |-- distributions.py          # Generadores de muestras (H_0 y H_a)
|   |-- simulation.py              # Estudio MC secuencial
|   |-- simulation_parallel.py     # Estudio MC paralelizado
|   |-- plotting.py               # Graficas de los resultados
|
|-- scripts/
|   |-- run_test1_simulation.py    # Script CLI secuencial
|   |-- run_test1_simulation_parallel.py # Script CLI paralelizado (recomendado)
|
```

```
|-- tests/
|   |-- test_statistics_ks.py      # Tests unitarios
|
|-- results/
|   |-- data/                     # CSVs generados
|   |-- figures/                  # PNG generados
|
|-- documento/                   # Informe LaTeX del proyecto
|-- documentacion/               # (este documento)
```

3 Módulos del paquete `src/`

3.1 `statistics_ks.py` — estadístico T_n y estimadores

Implementa la simetrización empírica y el estadístico T_n de forma vectorizada para evaluar muchos θ simultáneamente.

Función núcleo: `Tn_multi(sample, thetas)`. Para cada θ en `thetas` construye el grid $\{X_i\} \cup \{2\theta - X_j\}$ y evalúa

$$\text{diff}(x) = \frac{1}{2}(F_n(x) - 1 + F_n((2\theta - x)^-)),$$

usando `np.searchsorted`. El estadístico es $\sqrt{n} \cdot \sup_x |\text{diff}(x)|$.

```
1 def Tn_multi(sample, thetas):
2     x = np.asarray(sample, dtype=float)
3     n = x.size
4     x_sorted = np.sort(x)
5     th = np.asarray(thetas, dtype=float)
6
7     refl = 2.0 * th[:, None] - x_sorted[None, :]
8     grid = np.concatenate(
9         [np.broadcast_to(x_sorted, refl.shape), refl], axis=1
10    )
11
12    Fn = np.searchsorted(x_sorted, grid, side="right") / n
13    arg = 2.0 * th[:, None] - grid
14    Fn_minus = np.searchsorted(x_sorted, arg, side="left") / n
15
16    diff = 0.5 * (Fn - 1.0 + Fn_minus)
17    sup = np.max(np.abs(diff), axis=1)
18    return np.sqrt(n) * sup
```

Listing 1: Vectorización de T_n sobre múltiples θ

Estimadores del centro.

- `theta_median(sample)`: `np.median(sample)`.
- `theta_trimmed(sample, trim=0.1)`: `scipy.stats.trim_mean`.
- `theta_argmin(sample, n_grid=60)`: búsqueda en dos etapas
 1. Grid uniforme de 60 puntos en el rango muestral ampliado un 10 %.
 2. Refinamiento local con `scipy.optimize.minimize_scalar` (método de Brent) en el sub-intervalo del mejor punto del grid.

El diccionario ESTIMATORS mapea los nombres `.argmin`, `"median"`, `"trimmed"` a las funciones, permitiendo selección por cadena en el resto del paquete.

3.2 bootstrap_ks.py — procedimiento bootstrap

Implementa el algoritmo de Schuster–Barker:

1. Estima $\tilde{\theta}$ con el estimador elegido sobre la muestra original.
2. Calcula $T_{\text{obs}} = T_n(X; \tilde{\theta})$.
3. Construye el *soporte simetrizado* $\{X_i, 2\tilde{\theta} - X_i\}$ ($2n$ puntos con masa uniforme): esto es equivalente a muestrear de $sF_n(\cdot; \tilde{\theta})$.
4. Para $b = 1, \dots, B$:
 - Remuestra $Y_1, \dots, Y_n$ con reemplazo del soporte simetrizado.
 - Recalcula $\tilde{\theta}^*$ sobre la remuestra (mismo estimador).
 - Calcula $T_n^*(b) = T_n(Y; \tilde{\theta}^*)$.
5. p-valor bootstrap (con corrección de continuidad):

$$p = \frac{1 + \#\{T_n^*(b) \geq T_{\text{obs}}\}}{B + 1}.$$

```

1 def bootstrap_test_Tn(sample, estimator="argmin", B=500, rng=None):
2     if rng is None:
3         rng = np.random.default_rng()
4     est_fn = ESTIMATORS[estimator] if isinstance(estimator, str) else estimator
5
6     x = np.asarray(sample, dtype=float)
7     n = x.size
8     theta_hat = est_fn(x)
9     T_obs = Tn_statistic(x, theta_hat)
10    support = symmetrized_sample(x, theta_hat)
11
12    T_boot = np.empty(B, dtype=float)
13    for b in range(B):
14        y = rng.choice(support, size=n, replace=True)
15        theta_b = est_fn(y)
16        T_boot[b] = Tn_statistic(y, theta_b)
17
18    p_val = (1.0 + np.sum(T_boot >= T_obs)) / (B + 1.0)
19    return BootstrapResult(...)
```

Listing 2: Núcleo del bootstrap simétrico

El resultado es un `BootstrapResult` (dataclass) con: `statistic`, `p_value`, `theta_hat`, `boot_statistics`, `estimator_name`, `B` y la propiedad `reject` (`p_value < 0.05`).

3.3 distributions.py — generadores de muestras

Define el dataclass `DistSpec(name, sampler, theta_true, under_h0)` y factorías para cada distribución.

Factoría	Distribución	Centro θ
<code>uniform_h0(1.0, 3.0)</code>	Uniforme(1, 3)	2,0
<code>cauchy_h0(2.0, 1.0)</code>	Cauchy(loc=2, scale=1)	2,0
<code>normal_h0(0.0, 1.0)</code>	Normal estándar (opcional)	0,0
<code>gamma_ha(2.0, 1.0)</code>	Gamma($k=2, \theta=1$)	—
<code>weibull_ha(1.5, 1.0)</code>	Weibull($k=1,5$)	—
<code>pareto_ha(3.0, 1.0)</code>	Pareto/Lomax($a=3$)	—

Los catálogos por defecto se obtienen con `default_h0_specs()` y `default_ha_specs()`.

3.4 `simulation.py` — estudio Monte Carlo secuencial

Configurable vía la dataclass `SimConfig`:

```

1 @dataclass
2 class SimConfig:
3     sample_sizes: tuple[int, ...] = (20, 40, 80, 160)
4     estimators: tuple[str, ...] = ("argmin", "median", "trimmed")
5     B: int = 500
6     R: int = 300
7     alpha: float = 0.05
8     seed: int = 2026
9     progress: bool = True

```

`run_simulation(specs, config)` itera sobre (distribución $\times n \times$ estimador $\times$ réplica) y devuelve un DataFrame con una fila por réplica (columnas: `dist`, `under_h0`, `n`, `estimator`, `rep`, `statistic`, `p_value`, `reject`, `theta_hat`, `time_s`).

`summarize(df, alpha)` agrega por celda y calcula:

$$\widehat{\text{tasa_rechazo}}, \quad \bar{p}, \quad \overline{T_n}, \quad \bar{t}_s, \quad \text{SE} = \sqrt{\hat{p}(1 - \hat{p})/R}.$$

3.5 `simulation_parallel.py` — versión paralelizada

Usa `concurrent.futures.ProcessPoolExecutor` para distribuir las réplicas entre los núcleos disponibles. Cada tarea es independiente, con `seed` único = `config.seed + task_idx`, garantizando reproducibilidad.

Importante (Windows)

El script llamador **debe** usar `if __name__ == "__main__":` para que el *spawn* de procesos funcione correctamente.

Como las funciones `sampler` no son *picklables* si se construyen con *closures*, se mantiene un `_SPEC_REGISTRY` que mapea el nombre de cada distribución a la factoría que la genera. Cada worker reconstruye la `DistSpec` a partir de (módulo, factoría, kwargs).

Por defecto usa `os.cpu_count() - 1` workers. Devuelve el mismo DataFrame que la versión secuencial.

3.6 `plotting.py` — visualizaciones

Cinco funciones que reciben el DataFrame agregado y escriben PNG (300 dpi) en `results/figures/`:

Función	Salida
plot_type_i_error	tn_type_i_error.png
plot_power_curves	tn_power_curves.png
plot_runtime	tn_runtime.png
plot_power_vs_cost	tn_power_vs_cost.png
plot_pvalue_distribution_h0	tn_pvalue_h0.png

4 Scripts ejecutables

4.1 Versión secuencial

```
python scripts/run_test1_simulation.py # full
python scripts/run_test1_simulation.py --quick # B=199, R=80
```

4.2 Versión paralelizada (recomendada)

```
python scripts/run_test1_simulation_parallel.py # full
python scripts/run_test1_simulation_parallel.py --quick # rapido
```

Diferencias full vs -quick.

Parámetro	full	-quick
B (remuestras bootstrap)	500	199
R (réplicas MC)	300	80
Tamaños n	{20, 40, 80, 160}	{20, 40, 80}
Tareas totales	18 000	2 880
Tiempo aprox. (12 cores)	~15–20 min	< 1 min

5 Salidas generadas

5.1 Archivos de datos

- results/data/tn_simulation_raw.csv: una fila por réplica MC.
- results/data/tn_simulation_summary.csv: una fila por celda (distribución, n , estimador).

Columnas del CSV raw.

Columna	Descripción
dist	Nombre corto de la distribución
under_h0	True si la distribución es simétrica
n	Tamaño de la muestra
estimator	argmin, median o trimmed
rep	Índice de la réplica MC ($0, \dots, R - 1$)
statistic	T_n observado en esa réplica
p_value	p-valor bootstrap
reject	$p_value < \alpha$
theta_hat	Centro estimado
time_s	Tiempo del test (B remuestras incluidas)

Columnas adicionales del CSV `summary`. `reject_rate`, `mean_pvalue`, `mean_stat`, `mean_time_s`, `sd_time_s`, `n_rep` y `se_rate` (error estándar binomial $\sqrt{\hat{p}(1-\hat{p})/R}$).

5.2 Figuras generadas

- `tn_type_i_error.png`: tasa de rechazo bajo H_0 vs n , una sub-gráfica por distribución H_0 .
- `tn_power_curves.png`: potencia empírica vs n , una sub-gráfica por distribución H_a .
- `tn_runtime.png`: tiempo medio por test vs n , log-log, por estimador.
- `tn_power_vs_cost.png`: scatter potencia vs costo computacional.
- `tn_pvalue_h0.png`: histograma del p-valor bajo H_0 (debería ser $\sim U(0,1)$).

6 Tests unitarios

Ubicación: `tests/test_statistics_ks.py`. Cubre:

- `test_Tn_matches_reference`: la versión vectorizada coincide con la implementación naive $O(n^2)$.
- `test_Tn_vanishes_for_perfectly_symmetric_sample`: $T_n(\theta) = 0$ en una muestra simétrica exacta.
- `test_Tn_invariant_under_translation`: $T_n(X + c; \theta + c) = T_n(X; \theta)$.
- `test_theta_argmin_recovers_symmetry_center`: con $n = 400$ normales, el argmin recupera el centro a $\pm 0,25$.
- `test_theta_median_and_trimmed`: convergencia de mediana y media afeitada al centro real.
- `test_symmetrized_sample_is_symmetric_around_theta`: el soporte simetrizado es simétrico.
- `test_bootstrap_under_h0_p_value_uniform`: bajo H_0 la tasa de rechazo $\approx \alpha$.
- `test_bootstrap_has_power_against_gamma`: bajo H_a la potencia $> 0,4$ con $n = 80$.

Ejecutar todos los tests:

```
python tests/test_statistics_ks.py
```

7 Dependencias e instalación

7.1 Python

Probado con **Python 3.11.9**. Cualquier versión 3.10+ debería funcionar.

7.2 Paquetes

```
pip install numpy scipy pandas matplotlib
```

Versiones mínimas recomendadas: `numpy>=1.24`, `scipy>=1.10`, `pandas>=2.0`, `matplotlib>=3.7`. No se requieren paquetes externos adicionales.

8 Reproducir la simulación completa — guía paso a paso

1. Clonar el repositorio y posicionarse en la raíz:

```
git clone https://github.com/luistejon17/Proyecto2EstadisticaNP.git
cd Proyecto2EstadisticaNP
```

2. Instalar dependencias (opcionalmente en un entorno virtual):

```
python -m venv .venv
.venv\Scripts\activate      # Windows PowerShell
pip install numpy scipy pandas matplotlib
```

3. Verificación rápida con tests unitarios:

```
python tests/test_statistics_ks.py
```

4. Sanity-check con la versión `-quick` (menos de 1 min):

```
python scripts/run_test1_simulation_parallel.py --quick
```

Se generan los CSVs con sufijo `_quick` y todas las figuras.

5. Ejecución completa (B=500, R=300, ~15–20 min con 12 cores):

```
python scripts/run_test1_simulation_parallel.py
```

6. Resultados:

- Datos en `results/data/`
- Figuras en `results/figures/`
- Resumen impreso en consola (pivot table de tasas de rechazo).

Reproducibilidad

La semilla por defecto es `seed=2026`. En la versión paralelizada, cada réplica recibe un seed único derivado (`seed + task_idx`), por lo que los resultados son *exactos* entre corridas con la misma configuración, independientemente del orden de finalización de los workers.

9 Resumen de resultados obtenidos

Para referencia, la corrida completa (B=500, R=300) produce los siguientes valores (extracto):

Tasa de Error Tipo I bajo H_0 ($\alpha = 0,05$).

Distribución	Estimador	$n = 20$	$n = 40$	$n = 80$	$n = 160$
Cauchy(2,1)	argmin	0.030	0.023	0.053	0.050
Cauchy(2,1)	median	0.027	0.017	0.020	0.030
Cauchy(2,1)	trimmed	0.037	0.027	0.043	0.077
Uniforme(1,3)	argmin	0.017	0.033	0.017	0.033
Uniforme(1,3)	median	0.087	0.070	0.083	0.073
Uniforme(1,3)	trimmed	0.003	0.017	0.043	0.023

Potencia bajo H_a .

Distribución	Estimador	$n = 20$	$n = 40$	$n = 80$	$n = 160$
Gamma(2,1)	argmin	0.090	0.253	0.590	0.983
Gamma(2,1)	median	0.130	0.167	0.467	0.860
Gamma(2,1)	trimmed	0.027	0.140	0.543	0.963
Pareto(3,1)	argmin	0.290	0.683	0.980	1.000
Weibull(1.5,1)	argmin	0.043	0.183	0.407	0.910

Costo medio por test (s).

Estimador	$n = 20$	$n = 160$
median	$\approx 0,06$	$\approx 0,07-0,20$
trimmed	$\approx 0,20$	$\approx 0,20-0,50$
argmin	$\approx 1,0$	$\approx 1,4-3,1$

Lectura: **argmin** domina en potencia pero es 15–50× más lento que **median**; **median** es el más rápido y mantiene razonablemente el nivel; **trimmed** cae en una zona intermedia.

*Documento generado a partir del código presente en el repositorio del proyecto.
Para preguntas o errores, ver `CLAUDE.md` y `README.md` en la raíz.*